Exact rewards for fifteen single-day orienteering instances with time windows

This note gives a finite optimality table for fifteen of the 33 single-day Gavalas instances used by Gama and Fernandes [1], all from the t2xx group. The selection is by availability of completed finite upper certificates. The upper bounds have exact integer certificates. Each value is attained both with travel truncated downward to two decimals and with unrounded Euclidean travel. The result concerns fixed benchmark instances. It gives no improvement to a learning method or estimate of performance on generated tourists.

1 Model and values

Use the released single-day data [2], with the day specified in each file header. In these files the header field N counts customers, excluding the depot: there are $N+1$ vertex rows numbered $0, \dots, N$. The format description calls N the number of vertices, but the rows include this additional depot. Vertex 0 is the common depot. A customer j has coordinate $x_j \in \mathbb{R}^2$, nonnegative integral reward r_j, service duration $s_j \geq 0$, and window $[o_j, c_j]$. The depot has zero reward and duration. Its opening and closing times are o_0, c_0. All decimal input is interpreted exactly, without shifting or rescaling its meaning. The released `format_description_TTDP.txt` [2] specifies Euclidean distance rounded downward to two decimals. Accordingly, define travel time by

$$d_{ij} = \frac{\lfloor 100 \|x_i - x_j\|_2 \rfloor}{100}.$$

A route starts at 0 at time o_0, visits distinct customers, and returns to 0 by c_0. If it departs from i at time t, service at its next customer j starts at $b_j = \max\{t + d_{ij}, o_j\}$ and ends at $b_j + s_j$. Require $b_j \leq c_j$. Waiting is allowed. The objective is the sum of visited rewards.

The prose on p. 2 of [1] defines closing as when a visit must stop, which conflicts with its service-start test in Eq. (2.1). This note follows Eq. (2.1) and the released feasibility code [3]. Service need not finish before closing. Ten of the fifteen main truncated-travel witnesses finish service after a customer closes. For example, customer 17 in t203 starts at 1007.78, finishes at 1051.78, and closes at 1020. Eq. (2.2), p. 3, also imposes the direct-return condition

$$b_j + s_j + d_{j0} \leq c_0.$$

All supplied routes satisfy it. The upper certificates omit this condition, so they bound the larger route class defined above.

Theorem 1. *For each instance in Table 1, the final column is its optimal reward under the stated model, both with and without the direct-return condition. The same values are optimal if every d_{ij} is replaced by $\|x_i - x_j\|_2$, again with or without that condition.*

The completion-by-close optima for t203, t236 and t243 are not determined here.

2 Finite upper certificates

All coordinates, service durations and window endpoints in these files are integer multiples of 0.01. The checker rejects values outside that grid. For the exact checks, express times and coordinates in hundredths. In this section d_{ij}, s_j, o_j, c_j, t denote these integer-scaled quantities. If $X_j = 100 x_j$, then integer travel is

$$d_{ij} = \lfloor \|X_i - X_j\|_2 \rfloor .$$

Instance	Paper BKS	Best paper neural	Exact optimum
t201	185	191	191
t202	193	193	193
t203	179	181	182
t204	171	176	179
t207	201	200	201
t208	176	176	176
t218	155	152	155
t223	229	228	229
t227	159	156	159
t229	178	182	182
t236	175	177	180
t241	172	171	172
t242	180	180	180
t243	199	201	204
t250	200	201	201

Table 1: Raw single-day rewards; page numbers use arXiv v2 pagination. BKS is the historical column in Table B.7, p. 27, of [1], attributed to Gavalas et al. (2019) in its Section 4.1. The closing rule underlying those BKS values is not established here. The neural column is the maximum of global+ft in B.7 and model and model+as in B.1, p. 22, where present; the manifest lists each component. The main truncated witnesses satisfying completion by closing are t201, t202, t204, t207 and t208. These are historical benchmark scores, not generated-instance averages or current records.

This is computed as the integer square root of the integer squared distance. Let δ_{ij} be the shortest-path distance in this complete travel graph. Truncation can violate the triangle inequality, so shortest paths, rather than direct edges, supply optimistic reachability tests.

A label is (i, t, p, M): current vertex, departure time, collected reward, and remembered vertex set. Fix a neighborhood N_i containing i at each vertex. A permitted customer extension $j \notin M$ has

$$t' = \max\{t + d_{ij}, o_j\} + s_j, \qquad p' = p + r_j, \qquad M' = (M \cap N_j) \cup \{j\}.$$

It is omitted only if its service-start deadline fails or $t' + \delta_{j0} > c_0$. A label $(i, \tilde{t}, \tilde{p}, \widetilde{M})$ dominates (i, t, p, M) when

$$\tilde{t} \le t, \qquad \tilde{p} \ge p, \qquad \widetilde{M} \subseteq M.$$

This is the remembered-neighborhood relaxation of elementary paths [5]. These are established route-labeling ideas, not a new algorithm.

Lemma 1 (Resource bound)**.** *Put $m_i = \min_{k \neq i} d_{ik}$ and $w_j = 2(s_j + m_j)$. If an elementary suffix leaves i at time t, visits a nonempty customer set S, and reaches the depot by c_0, then*

$$\sum_{j \in S} w_j \le C(i, t) := 2(c_0 - t) - m_i - m_0. \tag{1}$$

Proof. For each edge of the suffix, the sum of the incident minima at its endpoints is at most twice the edge length. Sum over edges. Every interior customer contributes its minimum twice; the endpoints contribute m_i and m_0. Add twice the service durations. Travel plus service is at most the elapsed time; waiting can only increase elapsed time. This gives (1). □

For $C \ge 0$, let $K(C)$ be the maximum reward of a subset of all customers whose total weight w_j is at most C. This is an integer 0/1 knapsack, recomputed from the raw data. It may include already visited customers,

which only enlarges the bound. For a second bound, retain only customers outside M satisfying

$$a_j := \max\{t + \delta_{ij}, o_j\} \le c_j, \qquad a_j + s_j + \delta_{j0} \le c_0.$$

Let $F(i, t, M)$ be the fractional-knapsack optimum on those customers, with the same weights and capacity $C(i, t)$. Zero-weight customers are included first. Every elementary suffix of the fixed actual route that avoids M is admissible in both relaxations. This does not bound walks that collect a customer reward repeatedly. Thus its terminal reward is at most

$$B(i, t, p, M) = p + \min\{K(C(i, t)), \lfloor F(i, t, M) \rfloor\}. \tag{2}$$

The floor is valid because rewards are integers. If $C(i, t) < 0$, there is no nonempty suffix; use $B = p$. If $t + \delta_{i0} > c_0$, no completion exists. The empty suffix is checked separately by direct return.

A finite cover for a proposed upper $U \ge 0$ must satisfy three obligations. The root $(0, o_0, 0, \{0\})$ is dominated by a listed label or has bound at most U. Every permitted customer extension of every listed label has the same property. Every listed label that can return directly to the depot has reward at most U. All bounds in these obligations are recomputed, not read from the certificate.

Lemma 2 (Cover validity)**.** *These obligations imply that every elementary route has reward at most U.*

Proof. The zero-customer route has reward $0 \le U$. Fix a route with at least one customer. If a primitive bound closes the root, Lemma 1 and (2) give its reward at most U. Otherwise choose a dominating listed root label and follow the route's customers. At each stage keep a listed label at the same current vertex, with no later departure, no smaller reward, and memory contained in the actual visited prefix. Dominance preserves these conditions. The next actual customer is outside that memory. Its extension is therefore permitted; its memory remains within the enlarged actual prefix. An optimistic reachability test cannot exclude this feasible continuation.

If a primitive bound closes this extension, the remaining actual suffix avoids its memory. Lemma 1 and (2) bound the actual terminal reward by U. Otherwise choose a dominating listed label and continue along the fixed route. At its last customer the listed label can return no later than the actual route, so the terminal obligation applies. There are finitely many actual customers. Cycles among labels cannot invalidate this argument. Listed labels need not themselves be reachable. □

Proof of Theorem 1. The checker verifies all cover obligations: 225,910 labels and 11,439,464 permitted customer extensions. It also checks a distinct-customer route attaining each upper, including every direct-return condition. Lemma 2 proves equality for truncated travel. For each instance it checks a same-score unrounded route using rational upper enclosures of square roots. Since unrounded travel is no smaller on every edge, its feasible route class is contained in the truncated one. The upper and lower bounds therefore coincide for unrounded travel as well. □

3 Scope of the comparison

Eleven values already occur in the paper's BKS or neural columns. Only t203, t204, t236 and t243 exceed all those historical columns under the stated service-start model. Of these four main truncated witnesses, only t204 also satisfies completion by closing. Its excess therefore has a completion-feasible witness; the other three comparisons may depend on the closing-rule ambiguity. Four stored comparison routes attain the truncated optima for t202, t208, t218 and t227; the latter two also exceed the paper's neural scores. Their construction is not established by rescoring them. No learned improvement follows.

Exact decimal truncation differs from flooring a binary floating-point distance. Subsequent float32 scheduling can also change route acceptance. Passing a float32 test for the displayed witnesses would not exclude a different, better route accepted by that executable. Its operational optimum is unproved here. The theorem concerns the stated mathematical models only.

The fifteen cases are a subset of the 33 single-day cases used in the primary paper. The subset is defined by completed finite upper certificates. It is not a predictive holdout. No optimum for the other cases or generalization estimate follows. The finite covers supply the upper proofs for every row.

Established ng-route [5] and Pulse [6] methods predate these computations. Later MetaPNet work [4] reports further benchmark improvements, but its individual case table has not been obtained. No current-best, priority, speed, or method-novelty claim is made.

References

[1] R. Gama and H. L. Fernandes. *A Reinforcement Learning Approach to the Orienteering Problem with Time Windows*. Computers & Operations Research 133 (2021), 105357. arXiv:2011.03647v2.

[2] Gama and Fernandes, *optw_rl*, released benchmark and format description.
Commit `a21f70183b6a92bbfa71fa67b8cbb36b59888dcf`. Format description and benchmark source.

[3] Gama and Fernandes, *optw_rl*, same commit. Service-start feasibility code, lines 80–81.

[4] H. Kim and S. Barde. A meta-learning enhanced deep reinforcement learning approach for generalizing across orienteering problem with time windows. Transportation Research Part C 183 (2026), 105450. doi:10.1016/j.trc.2025.105450.

[5] R. Baldacci, A. Mingozzi and R. Roberti. New route relaxation and pricing strategies for the vehicle routing problem. Operations Research 59 (2011), 1269–1283. doi:10.1287/opre.1110.0975.

[6] D. Duque, L. Lozano and A. L. Medaglia. Solving the orienteering problem with time windows via the Pulse framework. Computers & Operations Research 54 (2015), 168–176. doi:10.1016/j.cor.2014.08.019.

Checker

Run `python3 -I check_hugo.py`. Python 3.9 or later suffices; only its standard library is used. Verification takes about 3–4 minutes on one core with Python 3.9.6. The script embeds the decimal instances, finite covers, routes, and historical table entries. It needs no network or external data. It checks the bounds, both travel conventions, completion-window outcomes, and the table comparisons. It rescores four stored comparison routes without asserting how they were constructed. Historical paper entries are transcribed inputs, not regenerated training results.

The readable `checker_manifest.json` lists routes, per-instance raw and cover SHA-256 hashes, label and extension counts, and the component neural scores with their column names. Regenerate it by adding `--manifest checker_manifest.json` to the command. Embedded hash checks establish internal consistency; they do not authenticate the instances against the external repository.

SHA-256 of `check_hugo.py`:
`7a47e62d8b5ab658e394ebe67bca5312`
`3f8f70f024d58b0a3c83ac808b682ec3`